Last Updated: June 2, 2026

File Structure

Key:
Purple = The starting point (AKA the README)
Red = Primary Key Files (recommended starting points for understanding the codebase and likely to be edited in the future)
Blue = Key Files (recommended starting points for understanding the codebase and likely to be edited in the future)
Gray = Can Ignore Initially (generated files, auxiliary files, files likely not to be edited in the future)
/ = File

Backend

app/

Backend API folder

- **data_cleaning/**

  Nested raw metadata

- **routes/**

  API routes for the frontend

  - **analytics.py**

    Routes for top items, top items (BigQuery), spend over time, data cleaning schema/dataset-config, period summary, item spend trends, external vendors, and bookstore items

  - **chatbot.py**

    Routes for the chatbot

- **explorer.py**

  Routes for the dataset explorer and its export button

- **feedback.py**

  Routes for bookstore inventory insights feedback form

- **insights.py**

  Routes for ML insights, including bookstore inventory insights, amazon demand insights

- **system.py**

  Routes for system health, status, checking available years in the Google Drive folder, and refresh

- **upload.py**

  Routes for projection mode (a previous dashboard feature)

- **services/**
  - **chatbot_prompts.py**

    Chatbot guided questions and guardrails

  - **chatbot_service.py**

    Chatbot guidance logic (Gemini/Vertex and fallback behavior)

- **analytics.py**

  Non-BigQuery dashboard analytics backend (item frequency, spend over time)

- **analytics_bookstore.py**

Bookstore-specific analytics backend

- **bigquery_service.py**
  BigQuery backend for most dashboard data and ML insights

- **data_config.py**
  Defines data cleaning schema. Used for column mapping (for frontend tables) and defines how each dataset's columns are normalized and which fields are available

- **dataset_explorer.py**
  Dataset explorer backend

- **drive.py**
  Google Drive sync backend. Used by the refresh pipeline to pull updated source files from the Drive folder

- **firebase.py**
  Firebase initialization backend. Used by upload metadata, external vendor CSV storage, and general data lookups.

- **main.py**
  Backend entry point. Defines API endpoints. Connects frontend and backend

data_cleaning/

Backend for files that clean the raw data

- **config/**
  Holds long lists and dictionaries used in data cleaning functions

  - **amazon_config.py**

Unnecessary columns, state map

- **cruzbuy_config.py**
  Unnecessary columns, non-item descriptions

- **onecard_config.py**
  Unnecessary columns, state map, merchant map, non-item descriptions

- **data/**
  Holds the clean and raw datasets locally (for testing or debugging)

  - **clean/**
    Cleaned datasets

  - **raw/**
    Raw datasets

    - **metadata.json**
      Stores Google Drive file timestamps used to detect whether source files changed

- **src/**
  Data-cleaning-specific source files

  - **clean_amazon.py**
    Amazon data cleaning

  - **clean_bookstore.py**
    Bookstore data cleaning

        - **clean_cruzbuy.py**
          CruzBuy data cleaning

        - **clean_onecard.py**
          OneCard data cleaning

        - **pipeline.py**
          Runs all cleaning scripts and returns the cleaned datasets

    - **data_mining.ipynb**
      Jupyter Notebook test file

.env.example
Template for how .env should look

requirements.txt
Python dependencies for backend

package-lock.json
Dependency lock (mostly unused)

test_drive.py
Google Drive connectivity test

firebase_client/

Backend files that upload cleaned datasets and summaries to Firebase

- **firestore.py**
  Writes dataset metadata and (optionally) rows to Firestore

- **generate_test_csvs.py**

Generates messy synthetic data for testing

- **pipeline.py**
  Entire upload pipeline. Uploads cleaned datasets (optional), metadata, and summaries (calls summaries.py)

- **storage.py**
  Uploads and timestamps cleaned datasets to Firebase Storage.

- **summaries.py**
  Uploads summaries (top values, spend over time, etc.) to Firestore

- **test_firestore.py**
  Local Firestore uploader tester

jobs/

Data pipeline jobs. Clean + upload pipeline runner. Also retrains the ML model and scrapes bookstore data

- **generate_mock_data.py**
  Generates multi-year fake CSVs and uploads them to Firestore (optional) for testing

- **retrain_models.py**
  Retrains ARIMA_PLUS ML model

- **run_bigquery_upload.py**
  Uploads cleaned data into BigQuery tables

- **run_cleaning.py**
  Data cleaning pipeline

- **run_firebase_uploads.py**
  Firebase uploads pipeline

- **run_full_pipeline.py**
  Runs entire pipeline (Both run_cleaning.py and run_firebase_uploads.py)

- **scrape_bookstore.py**
  Scrapes campus Bookstore prices, uploads them to Firestore, and creates a BigQuery price table

scripts/

- **upload_external_vendors.py**
  Uploads external_vendors_combined.csv to Firestore

Frontend

build/

Local frontend build for testing purposes. Generated by running `npm run build`.

node_modules/

Frontend dependencies

public/

Public assets

- **slugsmart.png**
  SlugSmart logo

src/

Source files

- ## components/

  Frontend (mostly Dashboard) components (what you see when viewing the website)

  - **figma/**

    Figma assets

  - **ui/**

    General UI assets

  - **AppHeader.tsx**
    Navigation Tab UI at the top of the website

  - **ChartCarousel.tsx**

    Carousel UI for Transactions Analytics Graphs, including the title, subtitle, next/previous arrow buttons, and the dot navigation

  - **Chatbot.tsx**
    Chatbot UI in the lower right corner of the website

  - **Dashboard.tsx**
    Main UI component. Includes Home Tab UI and Dataset Tabs UI

  - **ExternalVendorsPanel.tsx**
    Top External Vendors UI on Home Tab

  - **FeedbackModal.tsx**
    Feedback form UI that appears when you press “Flag Issue” on the Bookstore Inventory Insights card

- **HighImpactScatterPlot.tsx**
  High Impact Items scatterplot UI in the Transaction Analytics Graphs

- **InventoryInsights.tsx**
  Amazon Demand Insights/Bookstore Inventory Insights UI

- **ItemHistoryDrawer.tsx**
  UI for drawer card that appears when you press “Tap for Details” on insight card in Amazon Demand Insights/Bookstore Inventory Insights

- **ItemSpendTrendChart.tsx**
  Item Spend Trends UI in the Transactions Analytics Graphs

- **MetricsGrid.tsx**
  Metrics Grid UI for the Key Metrics in the SlugSmart Overview on the Home Tab

- **ProtectedRoute.tsx**
  Determines which page the user sees when first accessing the website. If logged in → Home Tab, else Login

- **PurchasePlan.tsx**
  Purchase Plan UI (the list created by adding insight items on the Amazon Demand Insights/Bookstore Inventory Insights via the “Add to Plan” button)

- **TabNavigation.tsx**
  UI for the button towards the top containing the Home, Amazon, Bookstore, CruzBuy, and Onecard tabs

- **TopItemDrilldownPanel.tsx**
  Detailed Breakdown UI under Top 5 Items in the Top Transaction Patterns chart

- **TopItemsChart.tsx**
  Top Transaction Patterns UI in the Transaction Analytics Graphs

- **TopItemsTable.tsx**
  Top Items/Current Campus Bookstore Inventory UI under the individual dataset tabs

- **TransactionsOverTimeChart.tsx**
  Total Spend Over Time UI in the Transaction Analytics Graphs

- ## context/

  React Context providers. Wraps the whole app and exposes state/hooks to many components

  - **AuthContext.tsx**
    Firebase authentication state

- ## hooks/

  React hooks

  - **useTopItems.ts**

    Fetches Top Items table data

  - **useSpendSeries.ts**

    Fetches Spend Over Time line graph data

  - **useTopPatterns.ts**

Fetches Top Transaction Patterns bar chart data

- **useOverallSummary.ts**
  Fetches Home tab summary data

- **useDatasetPreviews.ts**
  Fetches small preview lists for each dataset shown on the Home tab

- **use-mobile.ts**
  Detects whether the current screen is mobile-sized

- **lib/**

  Shared attributes for components and hooks

  - **dashboardTypes.ts**
    Shared dashboard types, tab mappings, chart helpers, dataset preview settings, and formatting utilities

  - **datasetConfig.ts**
    Frontend fallback dataset schema definitions used when backend schema data is unavailable

  - **categoryMapping.ts**
    Maps detailed source categories into broad dashboard filter categories

  - **firebase.ts**
    Initializes Firebase and exports auth, Google login provider, and Firestore access

- **collection.ts**

  Defines typed Firestore collection references for users and transactions

- **utils.ts**

  Utility for safely combining Tailwind/CSS class names

- **pages/**

  UI for website pages other than the Dashboard

  - **About.tsx**
    About page UI
  - **DatasetExplorer.tsx**
    Dataset Explorer UI
  - **Help.tsx**
    Help page UI
  - **Login.tsx**
    Login page UI
  - **Reports.tsx**
    Reports page UI

- **styles/**

  Shared CSS themes and variables used across the app

- **globals.css**
  UI baselines

- **types/**

  Shared Typescript interfaces used across the app

  - **index.ts**

    Shared TypeScript interfaces for transactions and user profile

- **utils/**

  Mock data generators for older/demo dashboard views

  dashboardData.ts

  Generates mock dashboard data

  mockData.ts

  Generates mock chart, product, and vendor data

- **App.tsx**

  Initial setup of dashboard, including routing, query caching, auth wrapper, layout, and startup prefetching

- **Attributions.md**
  Figma license and attributions file

- **index.css**
  Project-wide CSS endpoint

- **main.tsx**
  React entry point

- **vite-env.d.ts**
  Vite ambient type definitions

components.json
shadcn config file

index.html
Vite/React app shell entry point

package-lock.json
Locks frontend dependency versions installed by npm

package.json
Frontend dependency file

tsconfig.app.json
App-specific Typescript compiler settings

tsconfig.json
Typescript compiler settings

vite.config.ts
Vite build/dev server config file

Root

Non-backend/frontend files

.env
Local environment configuration variable for backend and frontend. Not important for understanding the code, but essential to running the app locally. Must be kept secret

.gitignore
Tells Git which files should be ignored when pushing to the repo

google-drive-service.json
Google service account credentials for Google Drive access. Not important for understanding the code, but essential to running the app locally. Must be kept secret

package-lock.json
Locks frontend dependency versions installed by npm

README.md
The first thing a new programmer should read and understand. Includes a project overview, components, user guide, programmer guide (setup, file structure, architecture, tech stack, testing, documentation) known issues, future work, and contributors

requirements.txt
Python dependencies for backend

serviceAccountKey.json
Firebase service account credentials for Firebase Storage/Firestore. Not important for understanding the code, but essential to running the app locally. Must be kept secret

vercel.json
Deployment configuration for Vercel